



Idempotência e Entrega Exactly-Once (na prática: at-least-once + dedup)

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company: Automação & Infraestrutura

Módulo: 05 · Sistemas Distribuídos, Mensageria e Eventos

Data: Agosto 2026

Status: **NÃO PUBLICADO** · Desenvolvido / Homologação pendente

Versão: 1.0

`exactly-once / dedupe / chave`

1. Contexto

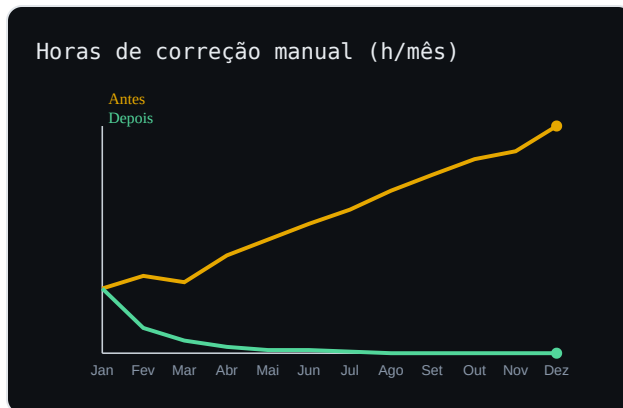
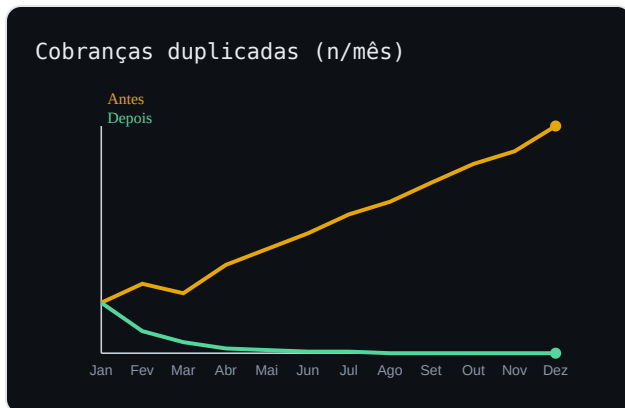
Quando você **desacopla** com mensageria, o mesmo evento pode chegar **duas vezes**. Rede caiu no `ack`? O broker reentrega. Consumer reiniciou? Ele reprocessa o último lote. Se cada entrega cobrar o cliente de novo, vira prejuízo e reclamação. O desafio é garantir que **processar 1x ou 5x dê o mesmo resultado**: é a **idempotência**.

ANALOGIA DA CONTA DE LUZ

Você recebe a conta e paga. O banco envia o comprovante 3 vezes por engano. Se cada comprovante disparasse um *novo* débito, você pagaria 3x. A conta tem um **código de autenticação**: o banco vê 'já paguei esse código' e ignora as cópias. A **chave de idempotência** é esse código: ela diz 'esse evento já foi processado, não faça de novo'.

2. Diagnóstico

Sem dedupe, reentregas do broker geram **cobranças duplicadas**, estoque negativo e e-mails repetidos. Times gastam horas apagando incêndio manualmente. O problema é estrutural: em sistema distribuído, *exactly-once* de ponta-a-ponta é impossível: só existe *at-least-once* + idempotência, ou *effectively-once*.



Raiz do problema: o consumer não distingue 'evento novo' de 'evento reentregue'. Falta uma **chave de idempotência** persistida e uma tabela de *já-processados* consultada antes de agir.

3. Solução

Dois pilares: (1) **idempotência no consumer**: toda ação só executa se a `idempotency_key` não existir numa tabela; (2) **Outbox Pattern**: em vez de escrever no banco E publicar no broker na mesma transação (que pode falhar no meio), você grava o evento numa tabela `outbox` dentro da **mesma transação** do negócio. Um *relay* depois publica o outbox no broker. Assim o evento nunca se perde nem duplica: banco e mensagem ficam atômicos.

```
# Outbox + idempotência (exemplo didático, Python)
import json, uuid, psycopg2

def registrar_venda(conn, venda):
    event_id = str(uuid.uuid4())
    with conn:
        # MESMA transação: banco + evento
        cur = conn.cursor()
        cur.execute("INSERT INTO venda VALUES (%s,%s)", (venda['id'], venda['valor']))
        cur.execute(
            "INSERT INTO outbox(event_id, tipo, payload, sent) VALUES (%s,'venda.criada',
            (event_id, json.dumps(venda)))

def consumer_cobrar(conn, key, venda):
```

```
with conn:
    cur = conn.cursor()
    cur.execute("SELECT 1 FROM processados WHERE idempotency_key=%s", (key,))
    if cur.fetchone():
        # reentrega -> ignora
        return "ignorado (dedupe)"
    cobrar(venda)
    # efeito só acontece 1x
    cur.execute("INSERT INTO processados(idempotency_key) VALUES (%s)", (key,))
    return "cobrado 1x"
```

4. Como funciona (pipeline)

Outbox + idempotência na cobrança

● FLUXO EM EXECUÇÃO

1

Transaction `DB.begin()`

Grava a venda E o evento na tabela `outbox` na mesma transação.

ok

2

Commit `db.commit()`

Se der erro, nada é gravado: consistência garantida.

ok

3

Relay `outbox.publish()`

Lê o outbox não enviado e publica no broker, marcando como enviado.

warn

4

Consumer `dedupe(key)`

Antes de cobrar, consulta `idempotency_key` na tabela de processados.

audit

5

Ack `consumer.ack()`

Só faz `ack` após gravar a chave: reentrega não cobra 2x.

ok

Falha no relay → evento fica no outbox até ser publicado; falha no consumer → reentrega cai no dedupe.

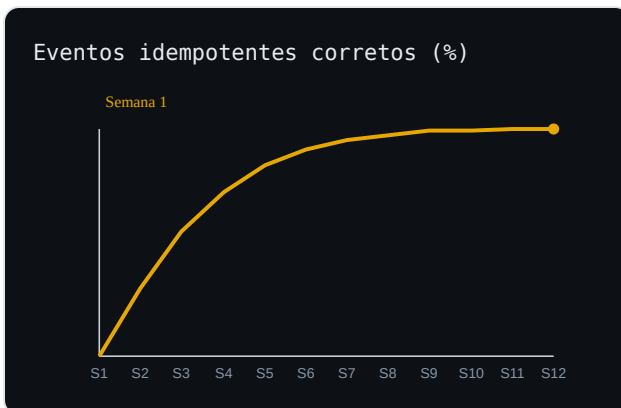
5. Antes vs Depois

CENÁRIO	ANTES (SEM DEDUPE)	DEPOIS (OUTBOX + DEDUPE)
Reentrega do broker	Cobra 2x	Ignorada (chave existe)
Falha na publicação	Evento perdido	Reprocessado do outbox
Consistência	Banco≠Mensagem	Atômica
Correção manual	~60h/mês	~0h/mês

6. Entregas

- **Migração** `001_outbox.sql`: tabela `outbox(event_id, payload, sent)`.
- **Relay** `outbox_relay.py`: publica pendentes no Kafka a cada 1s.
- **Consumer** `cobranca_idempotente.py`: dedupe por `idempotency_key`.
- **Teste** `test_reentrega.py`: injeta o mesmo evento 5x e afirma 1 cobrança.

7. Métricas



Meta: 100% de *effectively-once* nas cobranças, com zero débito duplicado mesmo sob reentrega contínua.

8. Decisões e tradeoffs

- **At-least-once do broker mais dedup no consumidor:** entrega exactly-once observacional para o negócio, porque exactly-once de ponta a ponta não existe em sistema distribuído.
- **Outbox transacional:** venda e evento gravados na mesma transação na tabela `outbox(event_id, payload, sent)`, então falha na publicação vira reprocessamento do outbox em vez de evento perdido. Custo: relay varrendo pendentes a cada 1s.
- **Dedupe por idempotency_key antes de agir:** o consumer consulta a tabela de processados antes de cobrar. Custo: store com TTL para não encher.
- **ACK só após gravar a chave:** a reentrega cai no dedupe, então o mesmo evento 5x gera 1 cobrança. A chave combina `event_id` com identificador do negócio para não colidir.

Fio condutor: a ordem importa (gravar chave, depois ACK), e cada escolha troca um risco financeiro (débito duplo) por um custo operacional conhecido (store com TTL, relay a cada 1s).

9. Impacto no negócio

Sem dedupe, reentregas geravam cobranças duplicadas e cerca de 60h por mês de correção manual. Com dedup a meta é zero duplicata e 100% dos handlers idempotentes, com correção próxima de 0h por mês. O teste que injeta o mesmo evento 5x e afirma 1 cobrança dá ao Financeiro previsibilidade: retry deixa de ser risco de débito duplo.

10. Referências de estudo

- **Curso:** "Event-Driven Architecture: From Theory to Practice" (Udemy).
- **Vídeo:** "What is Idempotency?" (YouTube, Hussein Nasser).
- **Doc oficial:** Apache Kafka Documentation, Exactly-once Semantics (kafka.apache.org).
- **Doc oficial:** PostgreSQL Documentation, INSERT ON CONFLICT (postgresql.org).

11. Status final

NÃO PUBLICADO : Desenvolvido e em homologação. Aguarda revisão antes de produção.